Lecture 15
I/O and Parsing

Reading and Writing with Java's Input/Output Streams and Parsing Utilities

Input/Output Basics

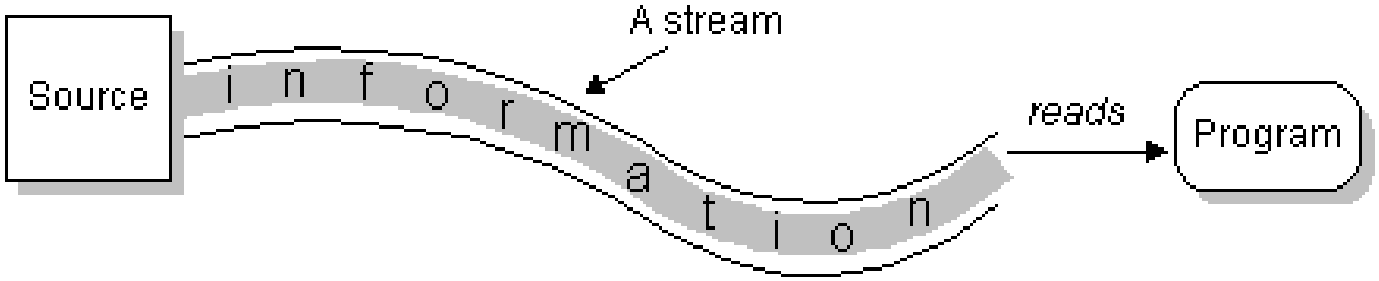
- Input/Output = I/O = communication between a computer program and external sources and destinations of information
- Involves *Reading* and *Writing*
  - Reading input from a source
  - Writing output to a destination
- Example Sources and Destinations:
  - Files
  - Network connections
  - Other programs

Java I/O Streams

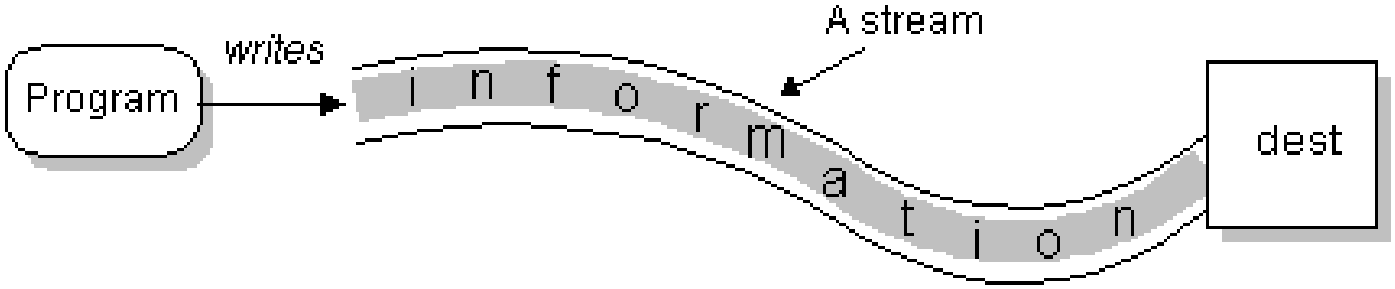
- Java uses an I/O system called *streams* (pioneered in C++)
- Java provides `java.io` package to implement *streams*
- *Streams* treat all external source and destinations of data the same way: as "streams" of information

Input vs. Output Streams

- **Reading from an Input Stream**



- **Writing to an Output Stream**



Byte vs. Character Streams

- **Byte Streams** are used to read and write data in binary format (1's and 0's)

  example data: images, sounds, executable programs, word-processing documents, etc.

- **Character Streams** are used to read and write data in text format (characters)

  example data: plain text files (txt extension), web pages, user keyboard input, etc.

Java Classes

- Package `java.io` offers classes to connect to streams

- To connect to a stream, instantiate a subclass of one of these abstract superclasses:

	input	output
byte	`InputStream`	`OutputStream`
character	`Reader`	`Writer`

Using a Stream Class

1. Open a stream by instantiating a new stream object
2. While more information to read/write, read/write that data
3. Close the stream by calling the object's `close()` method

Using a `Reader`

- Recall: a `Reader` is used to read a character input stream

- `Reader` offers these methods to read single characters and arrays of characters:

```
int read()
int read(char cbuf[])
int read(char cbuf[], int offset, int length)
```

- `Reader` is abstract so you must instantiate a subclass of it to use these methods

How to Read from a Text File

```
public void readFile() {
    FileReader fileReader = null;
    try {
        fileReader = new FileReader("input.txt");
        int c = fileReader.read();
        while (c != -1) {
            // cast c to char and use it
            c = fileReader.read();
        }
    } catch (FileNotFoundException e) {
        System.out.println("File was not found");
    } catch (IOException e) {
        System.out.println("Error reading from file");
    } finally {
       if (fileReader != null) {
        try { fileReader.close(); }
        catch (IOException e) { /* ignore */ }
       }
    }
}
```

Wrap in a `BufferedReader`

- `BufferedReader` has a `readLine()` method to read an entire line of characters efficiently

- Wrap a `Reader` with a `BufferedReader` by passing the `Reader` as a constructor argument

```
FileReader fr = new FileReader("myFile.txt");
BufferedReader br = new BufferedReader(fr);
```

- The `readLine()` method returns `null` when there are no more lines to read

Using `BufferedReader`

```
public void readFileWithBufferedReader() {
    BufferedReader bufferedReader = null;
    try {
        FileReader fr = new FileReader("input.txt");
        bufferedReader = new BufferedReader(fr);
        String line = bufferedReader.readLine();
        while (line != null) {
            // do something with line
            line = bufferedReader.readLine();
        }
    } catch (FileNotFoundException e) {
        System.out.println("File was not found");
    } catch (IOException e) {
        System.out.println("Error reading from file");
    }
    if (bufferedReader != null) {
        try { bufferedReader.close(); }
        catch (IOException e) { /* ignore */ }
    }
}
```

Writers

- `Writer` is an abstract class to write to character streams
- Offers `write` methods to write single characters, arrays of characters, and strings

```
void write(int c)
void write(char cbuf[])
void write(String str)
```

- `BufferedWriter` offers efficient writing and a `newLine()` method to insert a blank line
- Close writers with `close()` method when done

How to Write to a Text File

```
public void writeFileWithBufferedWriter() {
    BufferedWriter buffWriter = null;
    try {
        FileWriter fw = new FileWriter("output.txt");
        buffWriter = new BufferedWriter(fw);
        while (/*still stuff to write */) {
            String line = // get line to write
            buffWriter.write(line);
            buffWriter.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error writing to file");
    }
    if (buffWriter != null) {
        try { buffWriter.close(); }
        catch(IOException e) { /* ignore */ }
    }
}
```

Example: Copying Text Files

```
void copyFiles(String inFilename, String outFilename)
    throws FileNotFoundException {
  BufferedReader br = null;
  BufferedWriter bw = null;
  try {
    br = new BufferedReader(new FileReader(inFilename));
    bw = new BufferedWriter(new FileWriter(outFilename));
    String line = br.readLine();
    while(line != null) {
      bw.write(line);
      bw.newLine();
      line = br.readLine();
    }
  } catch (IOException e) {
    System.out.println("Error copying files");
  }

  if (br != null) {try {br.close();} catch(IOException e) {}}
  if (bw != null) {try {bw.close();} catch(IOException e) {}}
}
```

Reading from Keyboard Input

- Keyboard input is sent over a stream referred to as "standard" input
- Java "standard" input is the `InputStream` object `System.in` (a byte stream)
- To read characters over an `InputStream`, need to wrap it in an `InputStreamReader`
- To read line by line, wrap the `Input-StreamReader` with a `BufferedReader`

Reading from Keyboard Input

```
/**
 * Returns a line read from keyboard input.
 * Return null if there was an error reading the line.
 */
public void String readKeyboardLine() throws IOException {
   BufferedReader br = null;
   String line = null;
   try {
      br = new BufferedReader(new InputStreamReader(System.in));
      line = br.readLine();
   } catch (IOException e) {}

   if (br != null) {
      try { br.close(); }
      catch (IOException e) { /* ignore */ }
   }

   return line;
}
```

What We've Learned So Far

- Types of Streams
  - Input vs. output streams
  - Byte vs. character streams
- How to . . .
  - Read from text files
  - Write to text files
  - Read text from keyboard input
  - Use buffered streams
- You are left on your own to figure out how to use other streams

Intro to Parsing

- Programs often encode data in text format to store in files
- Programs later need to decode the text in the files back into the original data
- Process of decoding text back into data is known as *parsing*

Delimiters

- When data is stored in text format, *delimiter* characters are used to separate *tokens* of the data
- A list of first names stored separated by the '#' delimiter:
  `Greg#Kwame#Sonya#Bobby`
- Same list with a newline delimiter:

```
Greg
Kwame
Sonya
Bobby
```

StringTokenizer

- `java.util.StringTokenizer` separates `Strings` at the delimiters to extract tokens
- Default constructor will assume any whitespace (spaces, tabs, newlines) to be delimiters
- Second constructor accepts `String` of any delimiter characters
- `nextToken` method returns the next data token between delimiters in the text
- `hasMoreTokens` returns true if the text has remaining tokens

Using `StringTokenizer`

- Printing out every name from a file where names are delimited by whitespace:

```
public void printNamesFromFile(String filename) {
  BufferedReader br = null;
  try {
    br = new BufferedReader(new FileReader(filename));
    String line = br.readLine();
    while(line != null) {
        StringTokenizer st = new StringTokenizer(line);
        while(st.hasMoreTokens()) {
          System.out.println(st.nextToken());
        }
        line = br.readLine();
    }
  } catch (IOException e) {
    System.out.println("Error reading from file.");
  }
  if (br != null) { try { br.close(); } catch(IOException e) {} }
}
```

Text → Numbers

- Often necessary to parse numbers stored as text into Java primitives
- Wrapper classes for primitives provide static methods to do so

```
int Integer.parseInt(String s)
double Double.parseDouble(String s)
```

- Throw `NumberFormatException` if the specified `String` cannot be converted into the primitive

Putting it All Together

- File 1: `Employee_May.dat`

Format:

```
Name, SSN, Hourly Rate, Salary to Date
Paul Njoroge, 555-12-3456, 65, 20000
Evelyn Eastmond, 555-22-2222, 70, 30000
Peilei Fan, 555-33-4444, 60, 15000
Ethan Howe, 555-44-5555, 80, 40000
Naveen Goela, 555-66-8888, 75, 20000
. . .
```

- File 2: `Hours_June.dat`

Format: Consecutive integers, which are the number of hours each employee has worked during June. The integers have the same sequence as that of the employee records.

Content: `50 60 40 50 70 . . .`

What We Need to Do . . .

1. For each employee, multiply the hours worked by the hourly rate

2. Add this to the value of the salary to date

3. Write to a new file named `Employee_June.dat`, in the same format as `Employee_May.dat`, only it includes the updated, increased value of the salary to date.

- Create a StringTokenizer over the single line in the `Hours_June.dat` file

```
BufferedReader empReader = null;
String hoursLine = null;
try {
    empReader = new BufferedReader(
                    new FileReader("Hours_June.dat"));
    hoursLine = empReader.readLine();
} catch(IOException e) {
    System.out.println("Could not read Hours_June.dat");
}

if (empReader != null) {
    try { empReader.close(); }
    catch(IOException e) {}
}

if (hourseLine == null) // exit and report an error
StringTokenizer hoursST = new StringTokenizer(hoursLine);
```

- Opening and closing the streams to the employee files

```
BufferedReader mayReader = null;
BufferedWriter juneWriter = null;
try {
    mayReader = new BufferedReader(
                    new FileReader("Employee_May.dat"));
    juneWriter = new BufferedWriter(
                     new FileWriter("Employee_June.dat"));

    // On next slide, we add code to parse the May data,
    // do the salary calculation, and write the June data

} catch(IOException e) {
    System.out.println("Error with employee files");
}
if (mayReader != null) {
    try { mayReader.close(); } catch(IOException e) {}
}
if (juneWriter != null) {
    try { juneWriter.close(); } catch(IOException e) {}
}
```

Writing the June Data

```
String employeeStr = mayReader.readLine();

while(employeeStr != null) {
    StringTokenizer empST =
        new StringTokenizer(employeeStr, ",");
    String name   = empST.nextToken();
    String ssn    = empST.nextToken();
    double rate   = Double.parseDouble(empST.nextToken());
    double salary = Double.parseDouble(empST.nextToken());

    int hours  = Integer.parseInt(hoursST.nextToken());
    double newSalary = salary + hours * rate;
    juneWriter.write(name + "," + ssn + "," +
                     rate + "," + newSalary);
    juneWriter.newLine();
    employeeStr = mayReader.readLine();
}
```